

CSSCM 2026 COHORT

Numerical Python Hands-On Suite

Arrays and Multidimensional Arrays for Chemical Data Analysis

Director	Elliot S. Menkah (PhD)
Tutors	Emmanuel Oppong (MPhil)
	Obed Adu-Gyamfi (BSc)
Environment	Ubuntu + Python 3 + NumPy
Theme	Scientific Computing for Chemistry

Practical Objective

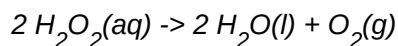
Use basic Python and NumPy to create, inspect, index, slice, transform and analyse one-dimensional and multidimensional chemical datasets, then solve a small system of linear equations arising from multicomponent chemical analysis.

Student file: `h2o2_analysis.py`

Run with: `python3 h2o2_analysis.py`

Background: Hydrogen Peroxide Decomposition

A chemistry laboratory is investigating the decomposition of hydrogen peroxide:



Assume first-order kinetics:

$$[\text{H}_2\text{O}_2]_t = [\text{H}_2\text{O}_2]_0 \exp(-kt)$$

$$\ln([\text{H}_2\text{O}_2]_t / [\text{H}_2\text{O}_2]_0) = -kt$$

$$k = -(1/t) \ln(C_t/C_0)$$

Time (s)	298 K	308 K	318 K	328 K
0	1.000	1.000	1.000	1.000
10	0.905	0.861	0.803	0.741
20	0.819	0.741	0.645	0.549
30	0.741	0.638	0.518	0.407
40	0.670	0.549	0.416	0.301
50	0.607	0.472	0.334	0.223
60	0.549	0.407	0.268	0.165

General instruction: Do not calculate values manually and type the answers into your program. Python must perform the calculations.

Part A - Creating One-Dimensional Arrays

Import NumPy using `import numpy as np`.

Create `time` containing 0, 10, 20, 30, 40, 50, 60 and `temperature` containing 298, 308, 318, 328.

Display each array, its number of elements, `.ndim`, `.shape`, and `.dtype`.

Part B - Creating a Multidimensional Array

Store all concentration measurements in one 2D NumPy array called `concentration`. Each row represents a time and each column a temperature.

Determine its number of dimensions, shape, total number of measurements, number of rows and columns. Confirm the shape is (7, 4).

Part C - Indexing

Without retyping values, extract: concentration at 20 s and 298 K; at 40 s and 318 K; at 60 s and 328 K; the first concentration measurement; and the final concentration measurement.

Remember that Python indexing starts from zero.

Part D - Array Slicing

Using slicing only, extract: all concentrations at 298 K; all at 328 K; all measurements at 30 s; all temperatures from 20-50 s; and the 308 K and 318 K columns only.

Use the pattern `array[rows, columns]`. Do not reconstruct arrays manually.

Part E - Array Arithmetic

For every temperature, calculate the percentage of H_2O_2 remaining after 60 s:

$$\% \text{ remaining} = (C_{60} / C_0) \times 100$$

Then calculate **% decomposed = 100 - % remaining**. Print both four-element arrays.

Part F - Statistical Operations

Using NumPy, determine the mean, minimum and maximum concentration at each temperature; the mean concentration at each time across all temperatures; and the overall mean.

Use `np.mean()`, `np.min()` and `np.max()`. Decide whether `axis=0` or `axis=1` is appropriate.

Part G - Finding the Rate Constant

Using the measurements at 60 s, calculate the rate constant for all four temperatures as one NumPy array.

$$k = -(1/t) \ln(C/C_0)$$

Use `np.log()`. Do not calculate each temperature separately.

Part H - Conditionals with Arrays

Classify each calculated rate constant using a for loop with `if`, `elif` and `else`.

Condition	Classification
$k < 0.015$	Slow
$0.015 \leq k < 0.025$	Moderate
$k \geq 0.025$	Fast

Print one line for each temperature, for example: 298 K : Slow.

Part I - Boolean Array Challenge

Without a loop, create a Boolean NumPy array identifying every location where $[\text{H}_2\text{O}_2] < 0.50 \text{ M}$.

Determine how many measurements satisfy the condition and extract the concentration values that satisfy it.

Part J - Repeated Experiments and Multidimensional Data

At 60 s the experiment is repeated three times. Create a 3 x 4 NumPy array from the data below.

Experiment	298 K	308 K	318 K	328 K
1	0.549	0.407	0.268	0.165
2	0.552	0.401	0.272	0.160
3	0.545	0.410	0.265	0.168

Calculate the mean final concentration and standard deviation for each temperature using `np.std()`.

Part K - System of Equations: Multicomponent Chemical Analysis

A laboratory mixture contains three absorbing species A, B and C. Measurements at three wavelengths produce:

$$2C_A + C_B + 3C_C = 9.5$$

$$C_A + 3C_B + 2C_C = 8.5$$

$$3C_A + 2C_B + C_C = 10.0$$

```
E = [[2, 1, 3],  
      [1, 3, 2],  
      [3, 2, 1]]  
A = [ 9.5 ]  
    [ 8.5 ]  
    [10.0 ]
```

`E @ C = A`

Create E and A as NumPy arrays. Solve for `C = [C_A, C_B, C_C]` using `np.linalg.solve(E, A)`.

Print each species concentration separately. Verify the solution with `E @ C` or `np.dot(E, C)` and compare it with the original absorbance vector.

Final Challenge - Build a Complete Chemistry Report

Modify the script so that its terminal output is clearly organised and includes the following sections.

```
=====
H2O2 DECOMPOSITION ANALYSIS
=====
Temperatures (K): [...]
Initial concentrations (M): [...]
Final concentrations (M): [...]
Percentage remaining: [...]
Rate constants (s^-1): [...]

Reaction classification:
298 K : ...
308 K : ...
318 K : ...
328 K : ...

=====
MULTICOMPONENT CHEMICAL ANALYSIS
=====
Species A concentration = ...
Species B concentration = ...
Species C concentration = ...
Calculated absorbances: [...]
Experimental absorbances: [9.5 8.5 10.0]

=====
ANALYSIS COMPLETE
=====
```

Rules and Submission Requirements

1. Use basic Python and NumPy in an Ubuntu terminal environment.
2. Do not use pandas, Excel or manually calculated answers.
3. Save the program as `h2o2_analysis.py`.
4. Run it with `python3 h2o2_analysis.py`.
5. Submit the Python script and a screenshot showing successful terminal execution.